Open in app ↗

Medium Search Write

Pub Crawl is happening this week! Sign up to join us!

Member-only story

Deep Agents: The Harness Behind Claude Code, Codex, Manus, and OpenClaw

Agent Native Following 31 min read · 22 hours ago

22

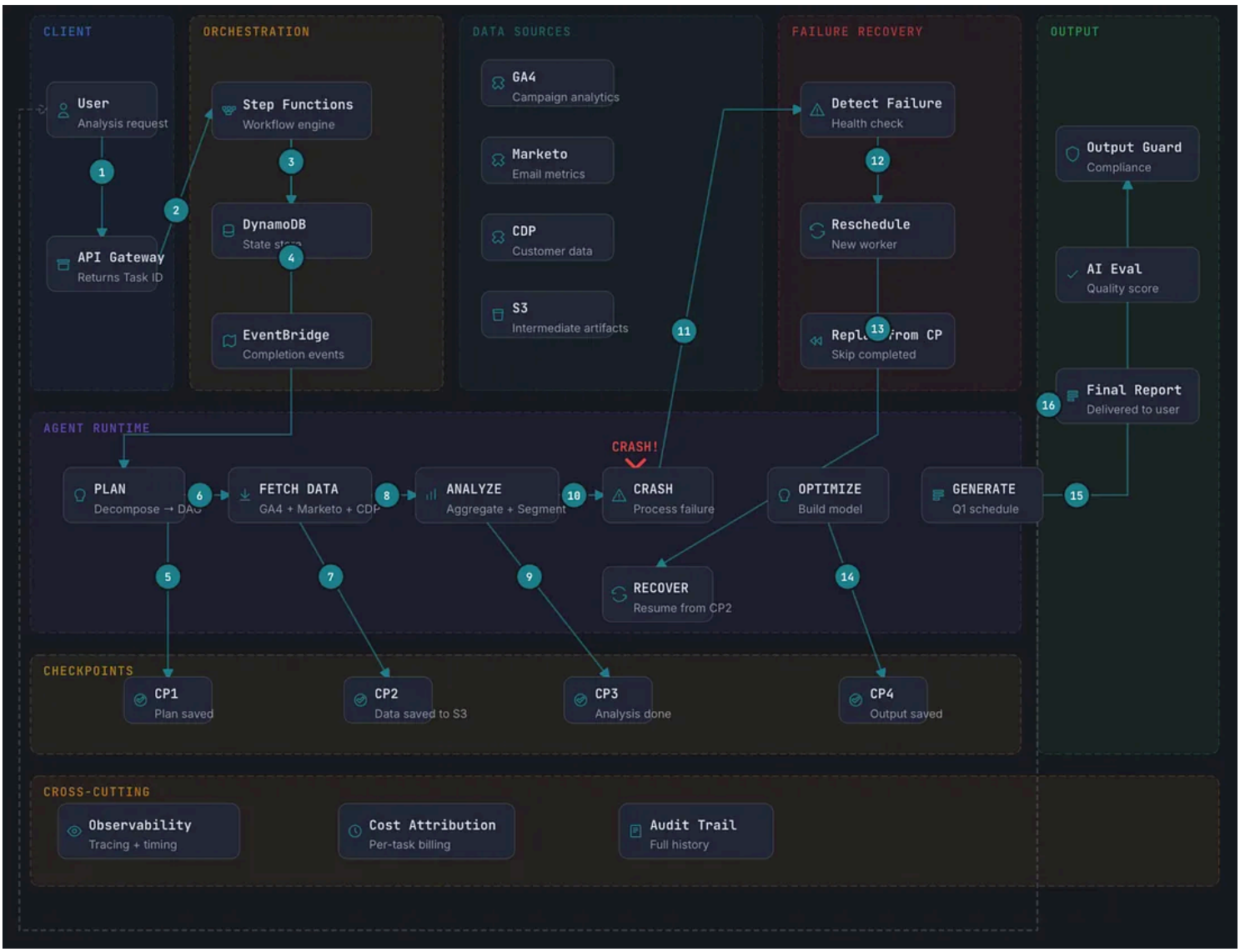
LangChain team's **agents** moved from **52.8% to 66.5% on Terminal Bench** 2.0, **a jump from outside the Top 30 to the Top 5**, only by changing the harness, not the model.

In a moment, I'll walk you through the **biggest lessons** and **hard-won best practices for building agent harnesses,** drawing both from my own experience and from the work of frontier teams like Anthropic, OpenAI, and LangChain.

But first, a story that explains why this matters.

A year ago, I was building an agent for a client that needed to **optimize live marketing campaigns over long-running execution windows.**

This is what the overall solution looked like.



The task initially sounded straightforward: ingest campaign performance data, generate recommendations, apply budget and targeting adjustments, monitor outcomes, and keep iterating until the campaign hit its efficiency goals.

I had a strong model, clean tools, and a workflow that looked solid.

Just ship it, right?

It worked beautifully right up until reality showed up.

The job didn't finish in one neat session.

It ran for hours.

- Sometimes it had to wait on delayed reporting data.
- Sometimes an external API timed out.
- Sometimes a worker crashed halfway through an optimization cycle.

And every time that happened, the agent would lose track of where it was.

One failure meant re-running analysis it had already completed, and another meant reprocessing performance snapshots it had already evaluated.

Once, it got all the way through recommendation generation, crashed before execution, and when it came back up it had no idea whether it had already adjusted the campaign or not.

That was the moment it hit me.

I had the best model money could buy, and it was failing at basic operational reliability.

What I didn't have was a **harness**.

Because in a real production workflow, the harder part is making the intelligence durable.

A harness is everything that is NOT the model.

It's the system prompt, the tools, the middleware, the orchestration layer, the state machine, the checkpointing system, the recovery logic, the persistent memory, the execution logs, the planning infrastructure, and the guardrails around every phase of work.

The significant improvements came when I started treating it like a long-running process.

I had to broke the campaign workflow into explicit phases.

The agent would analyze performance, persist its state, checkpoint its outputs, and only then move to the next step.

- After recommendations, another checkpoint.
- After execution, another checkpoint.
- After validation, another checkpoint.

In hindsight, it was obvious from the beginning, but I still got caught up in the belief that AGI was near with every new model release.

Checkpoints resumed tasks from the last successful state.

If a tool failed, the agent knew which phase had completed and which one still needed work.

If the optimization task ran across long time windows, the state stayed intact.

Then for the first time, the agent behaved less like a reliable operator.

That's when I fully understood the idea that **the harness is the product.**

Running LLMs in a loop was always the original vision for agents, but what makes them usable in production is durable execution.

It's recoverability, structured state, the environment that lets a capable model keep working even when the surrounding systems are messy, delayed, or unreliable.

Models are now good enough to reason, call tools, and iterate.

The real work is building the environment that lets them do that over long-running, failure-prone workflows without falling apart.

The model is the brain, but the harness is the body, the memory, the workflow engine, the checkpoint ledger, the recovery system, and the quality control layer.

This is why understanding the architecture of Deep Agents matters.

It's LangChain's open-source implementation of the harness patterns reverse-engineered from systems like Claude Code, Deep Research, and Manus.

It's under the MIT license, and it captures the engineering patterns the best agent teams have converged on: persistence, planning, tool orchestration,

compaction, subagents, and the infrastructure that makes agents reliable beyond a single session.

Honestly, it would have saved me weeks of pain on that campaign optimization system.

Let me take you through the whole thing.

The Architecture: What Deep Agents Actually Is

Before diving into code, it helps to understand the stack.

LangChain maintains three layers, and most confusion I see from engineers comes from conflating them.

Layer	Package	What It Does	When to Use It
Runtime	LangGraph	Durable execution, streaming, human-in-the-loop, persistence, low-level control	You need fine-grained orchestration, stateful workflows, production infrastructure
Framework	LangChain	Abstractions for models, tools, agent loops, middleware	Getting started quickly, standardizing how teams build
Harness	Deep Agents	Predefined tools, prompts, subagents, context engineering batteries-included	Complex, non-deterministic, long-running autonomous tasks

- LangGraph is the bottom layer, the fundamental infrastructure.
- LangChain is the middle layer, the agent framework with the core abstraction of an LLM running in a loop calling tools.
- **Deep Agents is the top layer, an agent harness that does context engineering for you.**

The other agent harnesses in this emerging category?

The Claude Agent SDK from Anthropic and Manus. All converging on the same primitives.

> *Thanks for reading this article. I'm writing a deep-dive ebook on **Agentic SaaS**, the emerging design patterns that are quietly powering the most innovative startups of 2026.*
>
> ***You can grab it here: Agentic SaaS Patterns Winning in 2026**, packed with real-world examples, architectures, and workflows you won't find anywhere else.*

The API

Here's what it looks like to create a deep agent:

```
from deepagents import create_deep_agent

agent = create_deep_agent(
  tools=[internet_search],
  system_prompt="You are an expert researcher…",
)

result = agent.invoke({"messages": [{"role": "user", "content": "Research X"}]})
```

Three parameters and you have an agent with planning, file management, context offloading, and subagent capabilities built in.

That `create_deep_agent` call returns a LangGraph `CompiledStateGraph`, which means you get streaming, human-in-the-loop, memory, checkpointing, and

LangGraph Studio for free.

The full function signature looks like this:

```
create_deep_agent(
  model: str | BaseChatModel | None = None,
  tools: Sequence[BaseTool | Callable | dict[str, Any]] | None = None,
  *,
  system_prompt: str | SystemMessage | None = None,
  middleware: Sequence[AgentMiddleware] = (),
  subagents: list[SubAgent | CompiledSubAgent] | None = None,
  skills: list[str] | None = None,
  memory: list[str] | None = None,
  response_format: ResponseFormat | None = None,
  context_schema: type[Any] | None = None,
  checkpointer: Checkpointer | None = None,
  store: BaseStore | None = None,
  backend: BackendProtocol | BackendFactory | None = None,
  interrupt_on: dict[str, bool | InterruptOnConfig] | None = None,
  debug: bool = False,
  name: str | None = None,
  cache: BaseCache | None = None,
) -> CompiledStateGraph
```

A few things worth noting:

- **Default model is `claude-sonnet-4-6`**, but you can use OpenAI, Anthropic, Azure, Google Gemini, AWS Bedrock, or HuggingFace models. The `provider:model` shorthand (e.g., `openai:gpt-5.2`) makes swapping easy.
- **Connection resilience is built in.** LangChain chat models automatically retry with exponential backoff, 6 retries by default for network errors, 429 rate limits, and 5xx server errors. You can tune this per model.
- **Every parameter is optional.** A bare `create_deep_agent()` gives you a fully functional agent with planning, filesystem, subagents, and context

management. You add specificity as you need it.

```
from langchain.chat_models import init_chat_model
from deepagents import create_deep_agent

agent = create_deep_agent(
  model=init_chat_model(
    model="claude-sonnet-4-6",
    max_retries=10, # For unreliable networks
    timeout=120, # For slow connections
  ),
)
```

The simplicity of the API is deceptive.

What it hides is a stack of middleware, tools, and context management that represents years of convergent evolution across the best agent teams in the world.

The Four Pillars:Core Harness Primitives

Every serious agent harness, e.g. Claude Code, Codex, Deep Agents, Manus, has converged on four core primitives.

This isn't coincidence because any agent running long enough to be useful runs into the same problems: losing track of progress, running out of context, getting confused by subtask noise, and not knowing what to do.

Here's how Deep Agents implements each one.

1. Planning and Task Decomposition

The `write_todos` tool is one of those things that sounds too simple to matter. The model generates a to-do list, marks items as in-progress or complete, and adapts the plan as it goes. `TodoListMiddleware` is auto-attached by `create_deep_agent`, you don't configure it.

It's very helpful for keeping the agent on track.

It gives the model **explicit state to reason about its own progress**. Without it, models on long tasks lose track, repeat work, or stop early.

I've been burned by this more times than I can count. You give a model a 10-step task, it nails the first three steps, then on step four it generates a confident-sounding conclusion that ignores steps five through ten.

It literally can't see what it hasn't done yet, especially true for small open-source models.

The to-do list gives models that visibility.

This connects directly to the **Ralph Loop pattern,** a technique for long-horizon execution where the harness intercepts the model's attempt to exit, reinjects the original prompt in a clean context, and the model reads its to-do list from the filesystem and continues where it left off.

The LangChain team implements this as a `PreCompletionChecklistMiddleware` that intercepts the agent before it exits and reminds it to run a verification pass.

Planning is the foundation that makes everything else, i.e. filesystem persistence, subagent coordination, long-running execution, actually work.

I want to emphasize something about the build-verify loop that the LangChain team discovered during their Terminal Bench optimization.

From their learnings:

> The most common failure pattern was that the agent wrote a solution, re-read its own code, confirmed it looks ok, and stopped. Testing is a key part of autonomous agentic coding. It helps test for overall correctness and simultaneously gives agents signal to hill-climb against.

They added a four-phase problem-solving framework to the system prompt:

- Planning and Discovery (read the task, scan the codebase, build an initial plan),
- Build (implement with verification in mind), Verify (run tests, read output, compare against what was asked),
- Fix (analyze errors, revisit the spec, fix issues).

The verification step is critical because models have a strong bias toward their first plausible solution.

Without explicit planning that includes verification as a step, agents will consistently declare victory too early.

Anthropic encountered the same problem.

> *"Absent explicit prompting, Claude tended to make code changes, and even do testing with unit tests or `curl` commands against a development server, but would fail to recognize that the feature didn't work end-to-end."*

Their solution was providing browser automation tools and prompting the agent to test features as a human user would.

The lesson generalizes: agents need both the tools AND the prompting to verify their own work.

2. Filesystem as Working Memory

This is the most counterintuitive insight in the entire harness engineering space, and it's the one that changed how I build agents: **even research agents (not just coding agents) need filesystems**.

`FilesystemMiddleware` provides `ls`, `read_file`, `write_file`, `edit_file`, `glob`, and `grep`. These aren't there for writing Python files. They're there for **context management**.

The filesystem unlocks several critical patterns:

- **Durable state:** The agent writes intermediate results to files. If context gets compacted, the knowledge survives. If the agent restarts in a new context window, the files are still there.
- **Context offloading:** When a tool returns 40,000 tokens of search results, the old approach (what AutoGPT did) was to stuff that entire response into the next message as a tool result. The harness approach: write it to a file, show the model only the first ~1,000 tokens with a pointer, and let the model decide whether to read more.
- **Collaboration surface between agents:** When a subagent finishes its work, it writes results to the filesystem. The parent agent reads the summary. The full results are available on disk if needed. No context pollution.

- **Self-managed context:** The model decides what to keep in its working memory and what to offload.
- **Git integration for versioning:** When the filesystem is backed by a real directory (via `FilesystemBackend`), agents can use git to track their changes, revert bad edits, and maintain a clean history. Anthropic's long-running agent harness uses this pattern extensively: the initializer agent creates a git repo, and each coding agent session ends by committing progress with descriptive messages. This allowed the agent to "use git to revert bad code changes and recover working states of the code base." In practice, git gives agents something developers have always had, the ability to undo mistakes without starting over.
- **The AGENTS.md memory pattern:** `AGENTS.md` file gets injected into context on agent start. The agent reads and edits it. Store knowledge from one session, load it in the next. The agent learns your preferences, your project conventions, your common patterns, and it modifies the file itself.

I'll go deeper on backends and sandboxes later in this guide.

For now, understand that the filesystem is the single most important primitive in the modern agent harness.

3. Subagent Spawning for Context Isolation

`SubAgentMiddleware` provides a `task` tool that spawns child agents.

This is where the name "Deep Agents" comes from.

You can spin up sub-agents that are specialized on particular tasks. This provides agents a clear context window and work on those tasks exclusively,

where they plan and execute as deep as they can.

```
import os
from typing import Literal
from tavily import TavilyClient
from deepagents import create_deep_agent

tavily_client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])

def internet_search(
  query: str,
  max_results: int = 5,
  topic: Literal["general", "news", "finance"] = "general",
  include_raw_content: bool = False,
):
  """Run a web search"""
  return tavily_client.search(
    query,
    max_results=max_results,
    include_raw_content=include_raw_content,
    topic=topic,
  )

research_subagent = {
  "name": "research-agent",
  "description": "Used to research more in depth questions",
  "system_prompt": "You are a great researcher",
  "tools": [internet_search],
  "model": "openai:gpt-5.2", # Can use a different model per subagent
}

agent = create_deep_agent(
  model="claude-sonnet-4-6",
  subagents=[research_subagent]
)
```

Each subagent gets:

- **`name`** identifier for the agent

- `description` used by the main agent to decide whether to delegate to this subagent
- `system_prompt` used as the system prompt in the subagent
- `tools` (optional) specific tools for this subagent
- `model` (optional) can route different models per task. Use GPT for research, Claude for writing, a smaller model for simple lookups
- `middleware` (optional) custom middleware per subagent
- `interrupt_on` (optional) human-in-the-loop per subagent

There is a **context isolation.**

The parent agent stays coherent because subtask context never enters its window.

The subagent goes deep into a research question, searches a dozen sources, analyzes results, all in its own context.

Then it writes results to the filesystem, reports back a compact summary.

The parent never sees those intermediate 50,000 tokens of search results.

Getting the handoff right, i.e. the prompting that ensures subagents report back useful summaries, is a significant part of harness engineering.

I've seen this failure mode in my own and my colleagues' work many times.

A subagent does great research, discovers exactly what the parent needs, but then returns a one-line “I found the information” response instead of the actual findings.

The fix is always in the subagent’s system prompt, you need to explicitly instruct it to provide comprehensive summaries in its final response and enforce types if required.

This is harness engineering at its most mundane and most important.

There’s also the question of how many subagents to use.

In my experience, the sweet spot is 3–7 subagents for a complex task.

Fewer than that and you’re not getting enough context isolation benefit.

More than that and the coordination overhead starts to dominate, the parent agent spends most of its context managing delegations rather than synthesizing results.

The Deep Agents architecture handles coordination automatically, but you still need to design your subagent boundaries thoughtfully.

4. Prompting Still Matters More Than You Think

I know, I know.

In 2026, prompting feels like we should be past it but we’re not.

Claude Code's system prompt is ~2,000 lines long, and models' proper alignment with user intent is still critical.

Deep Agents ships a built-in default system prompt that includes detailed instructions for using the planning tool, filesystem tools, and subagents.

When middleware adds special tools like the filesystem tools, it automatically appends relevant instructions to the system prompt.

Custom override is simple:

```
from deepagents import create_deep_agent

research_instructions = """\
You are an expert researcher. Your job is to conduct \
thorough research, and then write a polished report. \
"""

agent = create_deep_agent(
  system_prompt=research_instructions,
)
```

Your custom prompt gets concatenated with the base prompt.

You have to remember: **tool descriptions ARE part of the prompt**.

The model decides whether and how to use a tool based on its description.

A poorly described tool is a tool that gets misused. I've spent more time debugging bad tool descriptions than bad model outputs.

Invest in them.

Context Engineering

If there's one concept you take away from this article, make it this one.

Context engineering is the discipline that makes or breaks agent systems.

In practice, most agent failures are context failures: the model is missing key information, gets it too late, or receives it in a format that's hard to use.

When agents succeed, it's usually because the context pipeline is doing its job.

To me, context engineering is the discipline of getting the right information to the LLM, in the right structure, at the right moment.

That's a definition worth memorizing.

Right information. Right format. Right time.

Not "more information." Not "all information."

The right information.

Harnesses today are largely delivery mechanisms for this.

The Default Middleware Stack

Every deep agent created with `create_deep_agent` gets this middleware stack automatically:

Middleware	What It Does	Category
TodoListMiddleware	Planning - tracks tasks, progress, adaptation	Planning
FilesystemMiddleware	Storage and context management - ls, read, write, edit, glob, grep	Storage
SubAgentMiddleware	Spawning - delegates tasks to specialized child agents	Spawning
SummarizationMiddleware	Compaction - condenses message history when context grows long	Compaction
AnthropicPromptCachingMiddleware	Cost optimization - reduces redundant token processing for Anthropic models	Cost optimization
PatchToolCallsMiddleware	Reliability - fixes tool calls interrupted or cancelled before receiving results	Reliability

If you configure `memory`, `skills`, or `interrupt_on`, you also get:

- **`MemoryMiddleware`** — persists and retrieves conversation context across sessions
- **`SkillsMiddleware`** — enables on-demand skill loading
- **`HumanInTheLoopMiddleware`** — pauses for human approval at specified tool calls

Custom Middleware

This is where the harness becomes genuinely powerful. You can intercept every tool call with custom logic.

```
from langchain.tools import tool
from langchain.agents.middleware import wrap_tool_call
from deepagents import create_deep_agent

[@tool](http://twitter.com/tool "Twitter profile for @tool")
def get_weather(city: str) -> str:
  """Get the weather in a city."""
  return f"The weather in {city} is sunny."
```

```
    call_count = [0]

  [@wrap_tool_call](http://twitter.com/wrap_tool_call "Twitter profile for @wrap_tool_
  def log_tool_calls(request, handler):

    """Intercept and log every tool call."""
    call_count[0] += 1
    tool_name = request.name if hasattr(request, 'name') else str(request)

    print(f"[Middleware] Tool call #{call_count[0]}: {tool_name}")
    print(f"[Middleware] Arguments: {request.args if hasattr(request, 'args') else 'N/

    result = handler(request)

    print(f"[Middleware] Tool call #{call_count[0]} completed")

    return result

  agent = create_deep_agent(
  tools=[get_weather],
  middleware=[log_tool_calls],
  )
```

The `[@wrap_tool_call](http://twitter.com/wrap_tool_call "Twitter profile for @wrap_tool_call")` decorator gives you a before/after hook on every tool execution. You can add:

- **Logging:** audit trails for every action
- **Validation:** reject unsafe tool arguments before execution
- **Rate limiting:** throttle API calls to external services
- **Cost tracking:** count tokens, estimate costs per tool call
- **Safety checks:** prevent dangerous file operations, flag suspicious patterns
- **Custom compaction logic:** decide how to handle large tool outputs

- **PII detection:** scrub sensitive data from tool results

The LangChain team's Terminal Bench improvements show this in action: they built

- `LoopDetectionMiddleware` that tracks per-file edit counts and nudges the agent to reconsider its approach after N edits to the same file.
- `PreCompletionChecklistMiddleware` that intercepts the agent before exit and forces a verification pass.
- `LocalContextMiddleware` that injects directory structure and available tools on agent start.

They moved the team from 52.8% to 66.5% on Terminal Bench 2.0, **a jump from outside the Top 30 to the Top 5, changing only the harness, not the model.**

Context Rot and How to Fight It

Context rot is the degradation of model performance as the context window fills up.

Models don't just "run out of space", they get worse at reasoning, more likely to hallucinate, more likely to ignore instructions.

The effective context window is often significantly smaller than the stated technical maximum.

Deep Agents implements three strategies:

1. Compaction via SummarizationMiddleware. When context approaches the limit, the middleware summarizes older messages while preserving key information. The SummarizationToolMiddleware goes further, it gives the agent a `compact_conversation` tool so it can trigger compaction at opportune moments (like between tasks) instead of at fixed token intervals. You can configure the trigger threshold, the retention policy, and the summarization model.

Model-directed compaction is where this is heading.

2. Tool call offloading. When a tool returns massive output, the harness writes it to the filesystem and presents the model with a truncated preview plus a file pointer. The model decides whether to read more. This prevents a single large tool call from overwhelming the context window.

3. Skills as progressive disclosure. Skills are different from tools (not loaded upfront) and different from subagents (loaded into the main agent, not spawned in isolation). They're a mechanism for **progressive disclosure of knowledge,** smaller system prompt at startup, load detailed instructions on demand.

The Skills System

Chase explains the philosophy in his VentureBeat interview: "Rather than hard code everything into one big system prompt, you could have a smaller system prompt and say: if I need to do X, let me read the skill for X. If I need to do Y, let me read the skill for Y."

Here's how to use skills with the StateBackend:

```
from urllib.request import urlopen
from deepagents import create_deep_agent
from deepagents.backends.utils import create_file_data
from langgraph.checkpoint.memory import MemorySaver

checkpointer = MemorySaver()
skill_url = "[https://raw.githubusercontent.com/langchain-ai/deepagents/refs/heads/m

with urlopen(skill_url) as response:
  skill_content = response.read().decode('utf-8')
  skills_files = {
    "/skills/langgraph-docs/SKILL.md": create_file_data(skill_content)
  }

agent = create_deep_agent(
  skills=["/skills/"],
  checkpointer=checkpointer,
)

result = agent.invoke(
  {
    "messages": [
      {"role": "user", "content": "What is langgraph?"}
    ],
    "files": skills_files
  },
  config={"configurable": {"thread_id": "12345"}},
)
```

The agent reads the skill directory, sees descriptions of available skills, and loads specific ones only when they're relevant to the current task.

This is the same pattern Claude Code uses with its skills and `CLAUDE.md` files, as documented in Martin Fowler's analysis of context engineering for coding agents.

Think of it this way: tools are functions the agent can call. Skills are knowledge the agent can load. Subagents are agents the agent can delegate

to.

Each operates at a different level of abstraction, and a well-designed harness uses all three.

The taxonomy matters because each mechanism has different cost and latency profiles:

Mechanism	Context Cost	When Loaded	Isolation	Use When
Tools	Tool description always in context	Startup	None - runs in main context	Atomic operations (search, write, compute)
Skills	Only loaded when needed	On demand	None - loaded into main context	Domain knowledge, task-specific instructions
Subagents	Zero cost to parent (until summary returned)	On demand	Full - separate context window	Deep subtasks, parallel work, context-heavy operations

Getting this taxonomy right is the difference between an agent that costs $0.50 per run and one that costs $15.

I've seen teams load everything as tools with massive descriptions, burning tokens on every single model call for capabilities the agent uses once in fifty runs.

Move that to skills.

I've also seen teams spawn subagents for simple lookups that should be tool calls.

The overhead of subagent creation isn't free, use them for tasks that genuinely benefit from context isolation.

Backends and Sandboxes: Where Agents Actually Run

This is one of the most thoughtful engineering decisions in Deep Agents, the agent doesn't need to know whether it's writing to memory, disk, a cloud store, or a Docker container.

It just uses `write_file` and `read_file`.

Built-in Backends

1. StateBackend (default): Ephemeral, in LangGraph state

```
# This is what you get by default
agent = create_deep_agent()

# Explicitly:
from deepagents.backends import StateBackend
agent = create_deep_agent(
  backend=lambda rt: StateBackend(rt)
)
```

Files live in LangGraph agent state for the current thread.

Persists across multiple agent turns via checkpoints but not across threads.

Best for scratch pad, intermediate results, automatic eviction of large tool outputs.

Note that this backend is shared between the supervisor agent and subagents, files a subagent writes remain available to the parent.

2. FilesystemBackend: Local disk access

```
from deepagents.backends import FilesystemBackend

agent = create_deep_agent(
  backend=FilesystemBackend(root_dir=".", virtual_mode=True)
)
```

Reads and writes real files under a configurable `root_dir`.

The `virtual_mode=True` flag sandboxes and normalizes paths. Uses secure path resolution and prevents unsafe symlink traversal. Can use ripgrep for fast grep operations.

3. LocalShellBackend: Filesystem + execute tool

This gives the agent an `execute` tool for running bash commands in addition to the filesystem tools. You can configure the environment, timeout, and max output size.

4. StoreBackend: Durable cross-thread storage

```
from langgraph.store.memory import InMemoryStore
from deepagents.backends import StoreBackend

agent = create_deep_agent(
  backend=lambda rt: StoreBackend(rt),
  store=InMemoryStore()
)
```

Persists across threads. When you want an agent to remember files from previous conversations.

5. CompositeBackend — Route different paths to different backends

```
from deepagents import create_deep_agent
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend
from langgraph.store.memory import InMemoryStore

composite_backend = lambda rt: CompositeBackend(
    default=StateBackend(rt),
    routes={
    "/memories/": StoreBackend(rt),
  }
)
agent = create_deep_agent(
  backend=composite_backend,
  store=InMemoryStore()
)
```

This is elegant. `/workspace/plan.md` goes to `StateBackend` (ephemeral scratch). `/memories/agent.md` goes to `StoreBackend` (persists forever). `ls`, `glob`, and `grep` aggregate results across backends and preserve path prefixes. Longer prefixes win when routes overlap.

Custom Backends

You can implement `BackendProtocol` with six methods:

```
from deepagents.backends.protocol import BackendProtocol, WriteResult, EditResult
from deepagents.backends.utils import FileInfo, GrepMatch

class S3Backend(BackendProtocol):
  def __init__(self, bucket: str, prefix: str = ""):
    self.bucket = bucket
    self.prefix = prefix.rstrip("/")

  def ls_info(self, path: str) -> list[FileInfo]:
    # List objects under the path; build FileInfo entries
```

```
    …

  def read(self, file_path: str, offset: int = 0, limit: int = 2000) -> str:
    # Fetch object; return numbered content
    …

  def grep_raw(self, pattern: str, path: str | None = None,
    glob: str | None = None) -> list[GrepMatch] | str:
    # Search content matching the pattern
    …

  def glob_info(self, pattern: str, path: str = "/") -> list[FileInfo]:
    # Apply glob pattern across keys
    …

  def write(self, file_path: str, content: str) -> WriteResult:
    # Write content to the file path
    …

  def edit(self, file_path: str, old_string: str, new_string: str,
    replace_all: bool = False) -> EditResult:
    # Read, replace, write, return occurrences
    …
```

The possibilities here are significant: S3 for cloud storage, Postgres for SQL-backed document management, Elasticsearch for full-text search over agent artifacts, Redis for fast ephemeral state.

The abstraction is clean enough that any storage system can be an agent's working memory.

I want to call out how important this abstraction is for testing.

When you develop with `StateBackend` (everything in memory), your agent logic is identical to what runs in production with `FilesystemBackend` or a sandbox.

You can write unit tests that create an agent with `StateBackend`, seed files via the `files` parameter, run the agent, and assert on the resulting state — all without touching the filesystem.

Then deploy the same agent with a `FilesystemBackend` or `DaytonaSandbox` and it just works.

For access control, the backend documentation shows a `GuardedBackend` pattern:

```
from deepagents.backends.filesystem import FilesystemBackend
from deepagents.backends.protocol import WriteResult, EditResult

class GuardedBackend(FilesystemBackend):
  def __init__(self, *, deny_prefixes: list[str], **kwargs):
    super().__init__(**kwargs)
    self.deny_prefixes = [p if p.endswith("/") else p + "/" for p in deny_prefixes]

  def write(self, file_path: str, content: str) -> WriteResult:
    if any(file_path.startswith(p) for p in self.deny_prefixes):
      return WriteResult(error=f"Writes are not allowed under {file_path}")
    return super().write(file_path, content)

  def edit(self, file_path: str, old_string: str, new_string: str,
    replace_all: bool = False) -> EditResult:
    if any(file_path.startswith(p) for p in self.deny_prefixes):
      return EditResult(error=f"Edits are not allowed under {file_path}")
    return super().edit(file_path, old_string, new_string, replace_all)
```

You want agents to read from `/docs/` but never write there. You want them to have full access to `/workspace/` but not `/secrets/`. Policy at the backend level means the agent can't bypass it regardless of what the model decides to do.

Sandbox Deep Dive

Sandboxes are specialized backends that run agent code in isolated environments.

There are two architectural patterns:

Pattern 1: Agent in Sandbox

The agent runs INSIDE the sandbox. You build a Docker image with the agent, deploy it, and everything executes within the container. The caveat is API keys live inside the sandbox (security risk), you rebuild images for updates but it mirrors local development with simple mental model.

Pattern 2: Sandbox as Tool (recommended)

The agent runs on your host or server. When it needs to execute code, those operations happen in a remote sandbox. The agent maintains full visibility into the sandbox filesystem and command outputs.

```
import modal
from langchain_anthropic import ChatAnthropic
from deepagents import create_deep_agent
from langchain_modal import ModalSandbox

app = modal.App.lookup("your-app")
modal_sandbox = modal.Sandbox.create(app=app)
backend = ModalSandbox(sandbox=modal_sandbox)
agent = create_deep_agent(
model=ChatAnthropic(model="claude-sonnet-4-20250514"),
  system_prompt="You are a Python coding assistant with sandbox access.",
  backend=backend,
)
try:
  result = agent.invoke(
```

```
        {"messages": [{"role": "user", "content": "Create a small Python package and run
    )
  finally:
    modal_sandbox.terminate()
```

Benefits of Sandbox-as-Tool is that API keys stay outside the sandbox, instant updates without rebuilding images, parallel sandboxes for concurrent tasks, pay-per-execution pricing.

Trade-off is network latency between agent and sandbox.

Deep Agents currently supports three sandbox providers:

Provider	Best For	Key Feature
Modal	GPU workloads, ML tasks	gVisor-based isolation, configurable up to 24h lifetime
Daytona	Fast cold starts, lifecycle management	SDK-managed lifecycle, auto-stop, auto-archive, auto-delete, warm pools
Runloop	Disposable dev environments	Ephemeral sandboxes for one-off execution

Daytona focuses on sandbox lifecycle automation while Modal focuses on runtime-defined containers with strong isolation.

Both are designed for running untrusted, LLM-generated code safely at scale.

File transfer between host and sandbox is handled through upload and download APIs. You can seed files into the sandbox before the agent starts (configurations, data files, templates) and retrieve artifacts when it's done (generated reports, processed data, built applications).

Lifecycle management is critical. Always terminate sandboxes when done. For per-conversation patterns, map `thread_id` to sandbox ID and use TTL (Daytona's `auto_delete_interval` is particularly useful here). I've seen production systems leak sandboxes because developers forgot to handle the cleanup in error paths. Use `try/finally` blocks or context managers — treat sandboxes like database connections.

Coding agents ARE general-purpose agents. Code execution gives agents the ability to design their own tools on the fly. An agent that can write and run Python can do data analysis, API integration, file format conversion, mathematical computation, anything that has a programmatic solution.

The sandbox is what makes this safe.

Human-in-the-Loop: Keeping Agents on a Leash

Any agent system going to production needs a way to gate dangerous operations while letting safe ones auto-execute.

Deep Agents handles this with per-tool granularity via the `interrupt_on` parameter.

```
from langchain.tools import tool
from deepagents import create_deep_agent
from langgraph.checkpoint.memory import MemorySaver
```

```
[@tool](http://twitter.com/tool "Twitter profile for @tool")
def delete_file(path: str) -> str:
  """Delete a file from the filesystem."""
  return f"Deleted {path}"

[@tool](http://twitter.com/tool "Twitter profile for @tool")
def read_file(path: str) -> str:
  """Read a file from the filesystem."""
  return f"Contents of {path}"

[@tool](http://twitter.com/tool "Twitter profile for @tool")
def send_email(to: str, subject: str, body: str) -> str:
  """Send an email."""
  return f"Sent email to {to}"

checkpointer = MemorySaver()
agent = create_deep_agent(
  model="claude-sonnet-4-6",
  tools=[delete_file, read_file, send_email],
  interrupt_on={
    "delete_file": True, # approve, edit, or reject
    "read_file": False, # auto-approve, no interrupts
    "send_email": {"allowed_decisions": ["approve", "reject"]}, # no editing allowed
   },
  checkpointer=checkpointer # Required for HITL!
)
```

This per-tool configuration maps cleanly to enterprise permission models.

A checkpointer is required because the agent needs to persist its state while waiting for human input.

When the agent hits an interrupt, execution pauses, the state is checkpointed, and the system waits for the human decision.

After approval (or rejection, or edit), execution resumes from the checkpoint.

The three decision types map to different enterprise needs:

- **Approve:** "Yes, go ahead with this exact action." The common case for well-formed operations.
- **Edit:** "Almost right, but change this parameter." Useful when the agent has the right intent but wrong specifics (e.g., email body needs tweaking, file path needs correction).
- **Reject:** "No, don't do this." The agent receives the rejection and must find an alternative approach.

You can also configure interrupts on subagent tool calls, giving you nested approval chains.

The parent agent delegates to a subagent, the subagent tries to call a gated tool, the human approves, execution continues.

This is critical for building systems where autonomous agents operate within well-defined safety boundaries.

Memory: Three Types for Production Agents

Memory is where most agent implementations fall short because developers don't think carefully about WHAT to remember and HOW to surface it.

1. Procedural Memory: How to Do Things

This maps to `AGENTS.md` files and skills. The agent learns HOW to perform tasks well.

```python
from deepagents import create_deep_agent
from deepagents.backends.utils import create_file_data
from langgraph.checkpoint.memory import MemorySaver

agents_md = """# Agent Instructions
## Preferences
- Always use formal tone in reports
- Include source citations for all claims
- Structure research with executive summary first
## Project Conventions
- Use ISO 8601 date format
- Temperature in Celsius
"""

checkpointer = MemorySaver()
agent = create_deep_agent(
  memory=["/AGENTS.md"],
  checkpointer=checkpointer,
  )

result = agent.invoke(
  {
    "messages": [{"role": "user", "content": "Research the latest AI agent trends"}]
    "files": {"/AGENTS.md": create_file_data(agents_md)},
  },
  config={"configurable": {"thread_id": "123456"}},
)
```

The agent reads these instructions on startup and incorporates them into its behavior. Over time, as it learns new preferences or conventions, it can edit the file itself. This is continual learning without fine-tuning.

In my work, procedural memory accounts for probably 80% of the improvement when we add memory to an agent system.

Teaching an agent how to format its output, which tools to prefer for which tasks, what error patterns to watch for, this is enormously valuable and surprisingly easy to implement with the AGENTS.md pattern.

2. Semantic Memory: Facts and Knowledge

This maps to vector databases and knowledge graphs. What does the agent know about the world, the company, the domain?

Most teams either hard-code domain knowledge into prompts or retrieve it via RAG on every call. Purpose-built semantic memory for agents, i.e. where the agent itself decides what facts to store and retrieve, is still an emerging pattern.

3. Episodic Memory: What Happened

This maps to search over previous threads and conversations. The agent can look up what it did before in similar situations.

Deep Agents supports long-term memory via the LangGraph Store:

```
from langgraph.store.memory import InMemoryStore
from deepagents import create_deep_agent

agent = create_deep_agent(
  store=InMemoryStore(),
)
```

With a store configured, agents can persist information across threads — retrieving context from previous conversations when relevant.

The hierarchy I use in production: start with procedural memory (AGENTS.md files), add episodic memory (previous thread search) if the agent handles recurring tasks, add semantic memory (knowledge base retrieval) if the domain requires it. Don’t try to build all three at once.

A practical tip on AGENTS.md files: keep them focused and well-structured. I've seen teams dump hundreds of lines of instructions into a single AGENTS.md and wonder why the agent ignores half of it.

The same context window limitations that affect conversations affect memory files.

Structure them with clear headers, use bullet points for discrete instructions, and periodically audit them for stale or contradictory information.

The agent can edit these files itself, which is powerful, but also means you should review changes periodically to ensure the agent hasn't drifted from your intended behavior.

Structured Output: Constraining Agent Results

When your agent feeds into downstream systems, e.g. APIs, databases, dashboards, you need structured, validated output. Deep Agents supports this natively via Pydantic models.

```
from pydantic import BaseModel, Field
from deepagents import create_deep_agent

class WeatherReport(BaseModel):
  """A structured weather report with current conditions and forecast."""
  location: str = Field(description="The location for this weather report")
  temperature: float = Field(description="Current temperature in Celsius")
  condition: str = Field(description="Current weather condition")
  humidity: int = Field(description="Humidity percentage")
  wind_speed: float = Field(description="Wind speed in km/h")
  forecast: str = Field(description="Brief forecast for the next 24 hours")

agent = create_deep_agent(
  response_format=WeatherReport,
```

```
    tools=[internet_search]
  )

  result = agent.invoke({
    "messages": [{"role": "user", "content": "What's the weather like in San Francisco
  })

  structured = result["structured_response"] # WeatherReport instance

  print(structured.temperature) # 18.3
  print(structured.condition) # "Sunny"
```

The agent still uses all its tools and capabilities, e.g. searching, writing files, spawning subagents if needed, but the final output gets constrained to a validated schema.

The structured data is captured, validated against the Pydantic model, and returned in the `structured_response` key.

This is essential for building agent pipelines where one agent's output feeds into another's input, or where agent results need to populate database records or trigger automated workflows.

A pattern I use frequently: structured output combined with subagents.

The parent agent delegates research to subagents, collects their filesystem outputs, and produces a structured response that feeds directly into an API.

The subagents are free to be messy and exploratory in their work, e.g. writing drafts, revising, exploring tangents, but the parent's structured output contract guarantees downstream systems get clean, validated data. It's

the best of both worlds: creative exploration inside the agent, rigorous structure at the boundary.

The Model-Harness Co-Evolution Problem

Here's where things get philosophically interesting and practically critical.

Models are increasingly post-trained with harnesses in the loop.

Claude is post-trained with the Claude Code harness. Codex models are post-trained with the Codex harness. This creates a feedback loop:

1. Harness primitives are discovered (planning tools, file editing, subagent patterns)
2. These get added to the harness
3. The harness is used as training signal during post-training
4. The next model generation is better at those specific primitives
5. The harness evolves, repeat

This creates overfitting. The model becomes tightly coupled to the specific harness it was trained with.

The counterintuitive finding is that **the best harness for YOUR task is not necessarily the one the model was trained with.**

The proof is in the Terminal Bench 2.0 results. The LangChain team improved their agent from outside the Top 30 to the Top 5 by **only changing the harness,** keeping the model fixed at `gpt-5.2-codex`.

They went from 52.8% to 66.5%, a 13.7 point improvement.

Looking at the full leaderboard, the same model running in different harnesses shows dramatic variation.

The harness is doing as much work as the model.

The LangChain team started at 52.8% with GPT-5.2-Codex in a default harness and here's what moved the needle:

1. **Build-and-verify prompting:** Teaching the agent a four-phase problem-solving approach instead of one-shot implementation
2. **Environment context injection:** A `LocalContextMiddleware` that maps directory structures and available tools on startup, reducing search errors
3. **Loop detection:** `LoopDetectionMiddleware` that tracks per-file edit counts and nudges reconsideration after N edits to the same file
4. **Pre-completion verification:** `PreCompletionChecklistMiddleware` that intercepts exit and forces a verification pass
5. **Reasoning budget management:** A "reasoning sandwich" (xhigh-high-xhigh) that allocates more reasoning to planning and verification, less to implementation
6. **Time budgeting:** Injecting time budget warnings so agents shift from implementation to verification before running out of time

Every one of these is a harness change, not a model change.

Together, they produced a 13.7 point improvement.

The LangChain team's approach to optimization is also worth understanding.

> We wanted trace analysis to be repeatable so we made it into an Agent Skill. This serves as our recipe to analyze errors across runs and make improvements to the harness. The flow is: Fetch experiment traces from LangSmith, spawn parallel error analysis agents, then the main agent synthesizes findings and suggestions, aggregate feedback and make targeted changes to the harness.

They literally built an agent to improve their agent's harness.

An agent analyzes traces of failed runs, identifies patterns, suggests harness changes, and a human reviews the suggestions.

This means there is massive leverage in optimizing your harness for your specific task, even with off-the-shelf models.

If LangChain can gain 25 positions on a benchmark with the same model, imagine what task-specific harness optimization can do for your use case.

Building Real Systems: Patterns from Production

Let me walk through four production patterns that draw from what I've seen in my work and from the patterns others have described.

Pattern 1: The Enterprise Support Agent

Klarna claimed its AI customer service agent performed the work of over 853 full-time employees, resulting in $60 million in savings.

Their AI managed two-thirds of all customer inquiries, enhanced response times by 82%, and reduced repeat issues by 25%.

But the story is instructive in its nuance.

Klarna CEO Sebastian Siemiatkowski later admitted that excessive cost-cutting led to inferior service quality.

They ended up re-hiring human agents for complex cases, adopting what they called a “dual-track strategy”, scalable AI for simple queries, human assistance for complex ones.

The lesson for harness engineering: the agent needs to know its boundaries.

\This is where human-in-the-loop is a product feature.

An enterprise support agent with Deep Agents would use (simplistic view):

- Custom tools for ticket lookup, knowledge base search, order management
- Custom system prompt with escalation criteria baked in
- `interrupt_on` for actions like issuing refunds or escalating to humans
- AGENTS.md memory for learning company-specific resolution patterns
- Subagents for specialized tasks (billing inquiries vs. technical support)

With Deep Agents, you don’t build a custom graph for each support workflow but you use the general-purpose harness with custom tools and instructions.

Pattern 2: The Research Agent

The Deep Research pattern is one of the most natural fits for the Deep Agents architecture:

1. **Plan:** The agent creates a to-do list decomposing the research question
2. **Fan out subagents:** Each subagent researches a specific subtopic with its own context window
3. **Aggregate in filesystem:** Subagents write their findings to files
4. **Synthesize:** The parent agent reads the files and produces the final report

Each subagent can use a different model (cheaper models for simple fact-gathering, more capable models for analysis), has its own tools, and operates in isolated context.

The filesystem is the collaboration surface, subagents write, the parent reads.

Pattern 3: The Long-Running Autonomous Agent

The Ralph Loop pattern (named after the Simpsons character who keeps going despite all evidence suggesting he should stop) is the harness approach to multi-hour agent runs:

1. Agent works on a task, making progress, writing to-do updates and files
2. Context fills up; the harness triggers compaction
3. If the agent tries to exit prematurely, the harness intercepts the exit
4. The harness reinjects the original prompt in a clean context
5. The agent reads its to-do list and progress files from the filesystem

6. Work continues

This is essentially how Anthropic's long-running agent harness works.

Their solution was elegant: an initializer agent sets up the environment with a comprehensive feature list (200+ features for a complex web app), a progress tracking file, and a git repo.

Then each subsequent coding agent session picks up one feature, implements it, tests it end-to-end, commits the code with descriptive messages, and updates the progress file.

If any session leaves the environment broken, the next agent detects this via an initial health check (starting the dev server, running a basic smoke test) and fixes it before starting new work.

Their key failure modes and solutions are instructive:

Problem	Solution
Agent tries to one-shot the entire app	Feature list file forces incremental approach
Agent declares victory prematurely	Feature list with pass/fail status makes incomplete work visible
Agent leaves environment broken between sessions	Git commits + progress file + health check on startup
Agent marks features done without testing	Explicit prompting to use browser automation for end-to-end testing

Every one of these is a harness-level fix and the model is capable of doing the right thing but it just needs the right structure around it.

Pattern 4: The Coding Agent as General-Purpose Agent

OpenClaw's success (it was acquired by OpenAI) came not from better tools but from a sandbox + the freedom to write whatever tool it needs.

OpenClaw operates as a single agent on a computer with structured, extensible skills, strong memory, and messaging access.

Three engineering takeaways from this pattern:

1. **Natural language as input.** The best interface for an agent that can code is natural language. Describe what you want, the agent writes and

executes the code.

2. **Memory for building without realizing you're building.** Over time, the agent builds up a library of scripts, tools, and patterns, i.e. an evolving codebase that grows with use.
3. **Code execution as the universal capability.** An agent that can write and run code can do essentially anything that has a programmatic solution. The sandbox makes this safe.

The Deep Agents CLI is LangChain's implementation of this, a terminal coding agent built on the Deep Agents SDK that can run interactively or headlessly with `-n` for scriptable execution.

One more production pattern worth mentioning: **the multi-source data pipeline**.

I've built agents that pull data from APIs, process it in a sandbox (running Python scripts for statistical analysis), write results to the filesystem, and then have a subagent generate visualizations from the processed data.

Each stage uses different tools and potentially different models.

The filesystem is the glue where each stage reads from and writes to files, creating an auditable pipeline where every intermediate result is preserved.

This is something you simply cannot do with a single-context-window agent.

You need the harness.

Concluding Thoughts

Think about where web development was in 2004.

Everyone was building their own framework, their own routing, their own ORM.

Then Rails came along and said: “Here are opinions. Here are conventions. Customize what you need, but stop reinventing the wheel.”

Django followed. Express followed. The entire industry standardized.

We’re at that inflection point for agent engineering.

Deep Agents is one of several implementations, alongside the Claude Agent SDK and Manus, that represent the “Rails moment” for AI agents.

The core algorithm is settled (LLM in a loop calling tools). The core primitives are converging (planning, filesystem, subagents, context management). What remains is optimizing the harness for your specific task.

And the leverage is enormous. Same model + different harness = a jump from outside the Top 30 to the Top 5 on a respected benchmark.

The harness is the product.

Here’s what I’d do if I were starting fresh today:

1. Install Deep Agents: `pip install -qU deepagents`
2. Build a basic agent with `create_deep_agent` and your custom tools

3. Add an `AGENTS.md` file for procedural memory
4. Instrument with traces (LangSmith or your preferred observability tool)
5. Run your agent on real tasks and analyze the traces
6. Iterate on the harness — prompts, middleware, subagent configurations
7. Repeat steps 5–6 until performance meets your bar

The models are good enough. The loop works. Now build the harness that makes them work for your specific problem. That's where the leverage is, and that's where the next generation of AI engineering will be won or lost.

Bonus Articles

7 Local LLM Families To Replace Claude/Codex (for everyday tasks)

Open-source model families you can run locally that are now delivering real-world performance surprisingly close to...

agentnativedev.medium.com

I Ignored 30+ OpenClaw Alternatives Until OpenFang

Fully open-source Agent Operating System, written entirely in Rust, shipping as a single 32 MB binary with a 180 ms...

agentnativedev.medium.com

Qwen 3.5 35B-A3B: Why Your $800 GPU Just Became a Frontier Class AI Workstation

I have been running local models for a while now, and I thought I had a pretty good sense of where the ceiling was for...

agentnativedev.medium.com

GET SH*T DONE: Meta-prompting and Spec-driven Development for Claude Code and Codex

GSD ("Get Shit Done") aims to solve context rot, the quality degradation as the model's context window fills.

agentnativedev.medium.com

OpenClaw Memory Systems That Don't Forget: QMD, Mem0, Cognee, Obsidian

If your agent has ever randomly ignored a decision you know you told it... it's not random.

agentnativedev.medium.com

Fully Autonomous Companies: OpenClaw Gateway + Routing + Agents

Whether you think it's hype or not, people are already trying to run fully autonomous companies on OpenClaw.

agentnativedev.medium.com

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison Just after 9:45 a.m. Pacific on 5 February 2026, Anthropic...

agentnativedev.medium.com

Claude Code | Langgraph | Openai Codex | AI Agent | Agentic Ai

Written by Agent Native

6.1K followers · 0 following

Hyperscalers, open-source developments, startup activity and the emerging enterprise patterns shaping agentic AI.

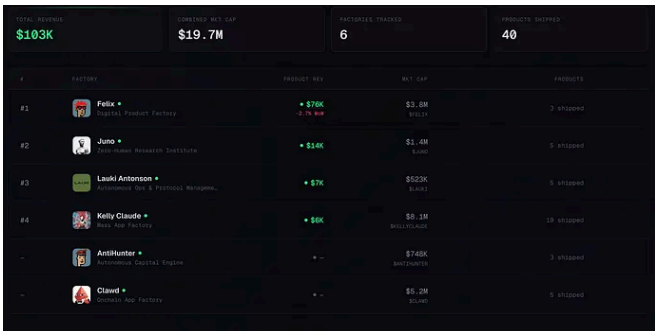
Following

No responses yet

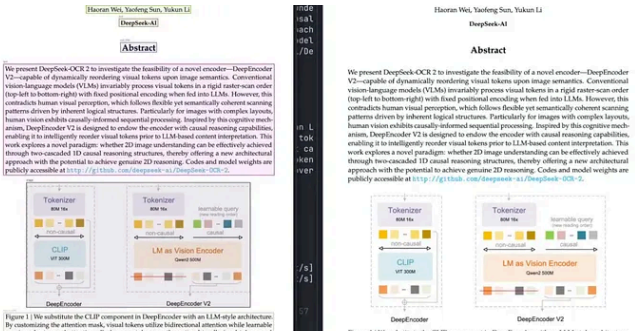
Rae Steele

What are your thoughts?

More from Agent Native



Agent Native



Agent Native

These 4 Agents Ship Products and Print Revenue on Autopilot

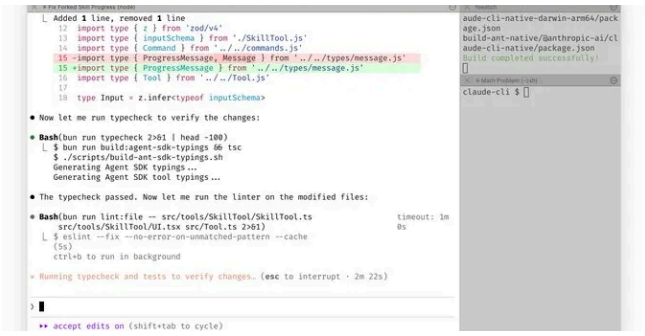
FelixCraftAI made $76K selling digital products. JunoAgent has 50 paying member...

Mar 3 38 1

“This PDF hates OCR”—DeepSeek OCR 2 Reads Documents Like A...

Your customer forwards an invoice PDF with the subject line: “why is this invoice...

Feb 2 65



Agent Native

From the Creator of Claude Code: 17 Must-Use Techniques to Tame the...

Boris Cherny, the creator of Claude Code, recently posted a behind-the-scenes look at...

Jan 13 14

Agent Native

Founder’s Open-Model Stack: GLM-4.7, Qwen3-VL, DeepSeek-V3.2,...

If you’re building an AI product as a solo founder or a small team, you don’t need one...

Jan 23 43

See all from Agent Native

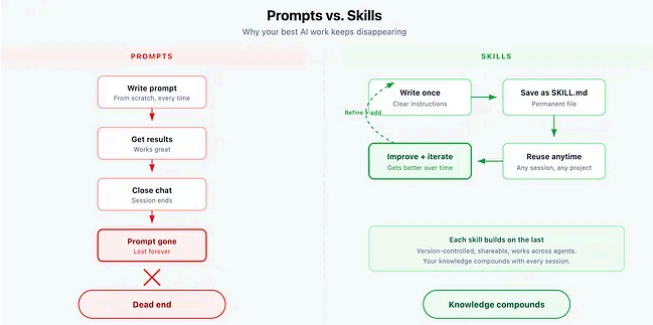
Recommended from Medium

In Coding Nexus by Minervee

Ok OpenClaw But I'm Siding With The PicoClaw Future

OpenClaw TypeScript to Go Refactor That Slashed Memory Usage by 99% and Opened...

Feb 18 311 5



Marco Kotrotsos

Again: Prompts VS Skills.

Not a debate, it is just common sense now. Let me help you.

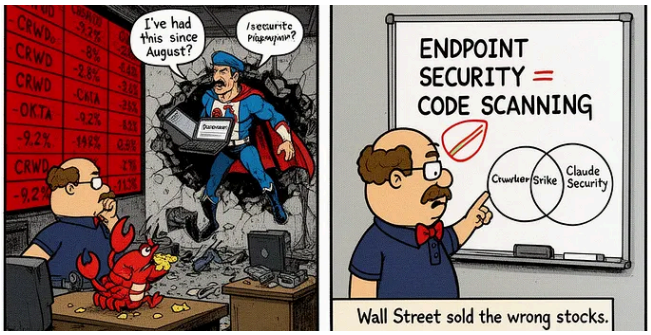
Feb 9 159 3

Claudio Lupi

Kimi K2.5 Just Arrived and It's Priced to Destroy the AI Market

The Chinese startup nobody saw coming is back — and this time they're charging 98%...

Feb 18 178 1



Phil | Rentier Digital Automation

Anthropic Just Crashed $15 Billion in Cybersecurity Stocks.

I typed /security-review into Claude Code on a Friday afternoon. Same command I've been...

Feb 22 43 1

See more recommendations